

CS336 Spring 2025 Assignment 2: Systems Writeup

Contents

1 Problem: benchmarking_script	1
2 Problem: nsys_profile	2
3 Problem: mixed_precision_accumulation	3
4 Problem: benchmarking_mixed_precision	4
5 Problem: memory_profiling	5
6 Problem: pytorch_attention	6
7 Problem: torch_compile	6
8 Problem: flash_forward	6
9 Problem: flash_backward	6
10 Problem: flash_benchmarking	6
11 Problem: distributed_communication_single_node	6
12 Problem: naive_ddp	6
13 Problem: naive_ddp_benchmarking	6
14 Problem: minimal_ddp_flat_benchmarking	6
15 Problem: ddp_overlap_individual_parameters	6
16 Problem: ddp_overlap_individual_parameters_benchmarking	6
17 Problem: ddp_overlap_bucketed	7
18 Problem: ddp_bucketed_benchmarking	7
19 Problem: communication_accounting	7
20 Problem: optimizer_state_sharding	7

1 Problem: benchmarking_script

(a) Deliverable

Implemented in `cs336_systems/benchmark.py`. The script initializes the model from CLI hyperparameters, creates random input data, runs warmup steps, then measures forward-only or forward+backward timing with `timeit.default_timer()` and `torch.cuda.synchronize()`.

(b) Forward/backward timings (5 warmup, 10 measured steps)

Model	Fwd Mean (ms)	Fwd Std (ms)	Bwd Mean (ms)	Bwd Std (ms)	Step Mean (ms)	Step Std (ms)
small	105.303	38.286	62.392	30.468	170.464	38.24
medium	184.545	38.394	85.389	23.722	288.648	25.34
large	339.047	2.485	156.582	1.572	499.472	3.68
xl	733.624	42.155	313.152	1.035	1099.266	5.91
2.7b	838.264	9.618	457.806	1.187	1307.808	24.51

Table 1: Part (b) benchmark results on CUDA (A100 40GB).

Forward and backward time increase with model size. Variability is generally low for larger models in this run (especially backward), while small/medium show higher standard deviation.

(c) Effect of fewer/no warmup steps

Model	Warmup 0		Warmup 1		Warmup 2		Warmup 5	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
small	225.844	138.148	169.072	36.043	176.863	30.199	170.464	38.242
medium	344.245	181.775	294.368	34.751	295.297	25.260	288.648	25.349
large	596.205	239.325	570.078	35.475	518.251	32.955	499.472	3.688
xl	1186.364	289.365	1077.503	31.389	1092.626	12.791	1099.266	5.916
2.7b	1402.911	225.944	1321.353	34.838	1319.389	37.589	1307.808	24.517

Table 2: Part (c): step total timing (ms), 10 measured steps, varying warmup count.

Without warmup ($w=0$), both average step time and variance are much worse across sizes. Using 1–2 warmup steps substantially improves stability, but can still differ from the 5-step baseline because one-time startup effects (kernel autotuning/selection, allocator behavior, cache setup, lazy initialization) may not be fully amortized yet.

2 Problem: nsys_profile

Profiling Coverage

We profiled with `nsys` at context lengths {128, 256, 512, 1024} for each model size {small, medium, large, xl, 2.7b}, using one profiled step after warmup. As allowed by the handout, several larger configurations ran out of memory and are noted below.

Model	128	256	512	1024
Forward-only				
small	yes	yes	yes	yes
medium	yes	yes	yes	yes
large	yes	yes	yes	OOM
xl	yes	yes	yes	OOM
2.7b	yes	yes	OOM	OOM
Train step (fwd+bwd+AdamW)				
small	yes	yes	yes	yes
medium	yes	yes	yes	yes
large	yes	yes	yes	OOM
xl	OOM	OOM	OOM	OOM
2.7b	OOM	OOM	OOM	OOM

Table 3: Nsight coverage; OOM indicates CUDA out-of-memory during warmup/profiled step.

(a) Forward-pass time and comparison to Python timers

Model (ctx=128)	Nsight forward (ms)	Part 1.1.3 forward mean (ms)	Delta (ms)
small	201.262	105.303	+95.959
medium	406.419	184.545	+221.874
large	781.562	339.047	+442.515
xl	1298.431	733.624	+564.807
2.7b	1209.294	838.264	+371.030

Table 4: Nsight forward-pass NVTX timing vs. Part (b) Python benchmark timings.

The Nsight forward-pass times are substantially higher than the Python benchmark values in this run. This mismatch is expected because Nsight instrumentation introduces overhead and here we profiled single iterations rather than averaging 10 timed steps like in Part (b).

(b) Dominant CUDA kernel and invocation count

In the forward pass, the highest cumulative-GPU-time kernel is generally a GEMM kernel (for example, `ampere_sgemm.128x64_tn` for small/medium/large at context length 128). For small@128 forward, this top kernel was invoked 25 times in one forward pass. The dominant kernel is not always the same once backward is included (e.g., small@128 train-step is dominated by a different GEMM variant), so the answer is configuration-dependent.

Here, a name like `ampere_sgemv_128x64_tn` denotes an Ampere-optimized single-precision matrix-multiplication kernel; `128x64` indicates the tile shape, and `tn` indicates transpose/non-transpose layout choices for the GEMM operands.

(c) Non-matmul kernels with non-trivial runtime

Beyond GEMMs, non-trivial forward runtime comes from elementwise and reduction kernels, including direct copy/masked fill/indexing and reduction-style kernels used by normalization and softmax-related computation. These appear consistently in the top non-GEMM entries across model sizes.

(d) Training-step vs inference kernel mix

On configurations where both modes fit in memory, matrix-multiplication kernels account for about 80.0% of CUDA kernel time in forward-only runs versus about 71.4% in full training steps. So compared to inference, training shifts more time into non-matmul work (backward elementwise/reduction kernels and optimizer multi-tensor update kernels).

(e) Softmax vs matmul within self-attention

Using an additional attention-annotated forward profile (medium@128), the summed attention softmax runtime was 48.61 ms, while attention matrix multiplications were 30.31 ms (QK) + 21.94 ms (PV) = 52.25 ms. This runtime gap is much smaller than the FLOP gap (matmuls are far higher FLOPs), consistent with softmax and related steps being more memory-bound and less compute-efficient than GEMMs.

3 Problem: mixed_precision_accumulation

(a) Accumulation experiments

We evaluated the four accumulation snippets from the prompt and obtained:

Case	Final value of <code>s</code>
<code>float32 accumulator + float32 addend</code>	10.0001335
<code>float16 accumulator + float16 addend</code>	9.9531250
<code>float32 accumulator + float16 addend</code>	10.0021362
<code>float32 accumulator + cast(float16 addend→float32)</code>	10.0021362

Table 5: Observed outputs from the four mixed-precision accumulation snippets.

(b) Why this happens

Accumulating directly in FP16 is much less accurate because FP16 has far fewer mantissa bits, so repeated additions incur larger rounding error. Using an FP32 accumulator significantly improves stability, but if each addend is first quantized to FP16 then that quantization error is still present, which is why cases (3) and (4) match each other and remain less accurate than pure FP32 accumulation.

4 Problem: benchmarking mixed_precision

(a) Dtypes under FP16 autocast

Assuming model parameters are initialized in FP32 and training is done on CUDA with `torch.autocast(..., dtype=torch.float16)`:

- Model parameters inside autocast: FP32 (autocast does not permanently cast parameters).
- Output of `ToyModel.fc1`: typically FP16.
- Output of `ToyModel.ln`: typically FP32 (layer norm is on autocast’s FP32-safe path).
- Model logits (`ToyModel.fc2` output): typically FP16.
- Loss: typically FP32 for common PyTorch loss ops under autocast.
- Gradients: FP32 (matching FP32 parameter storage dtype).

(b) Why layer norm is treated differently

Layer norm computes per-feature statistics (mean/variance) and involves reductions plus subtract/square/rsqrt operations that are numerically sensitive to precision. FP16 can lose significant accuracy (and may overflow/underflow more easily), so running layer norm in FP32 is safer. BF16 has a wider exponent range than FP16 (so overflow is less problematic), but it still has limited mantissa precision, so FP32 treatment of normalization remains helpful for stability.

(c) FP32 vs BF16 mixed-precision timings

Model	Fwd FP32 (ms)	Fwd BF16 MP (ms)	Bwd FP32 (ms)	Bwd BF16 MP (ms)	Step FP32 (ms)	Step BF16 MP (ms)
small	87.552	110.179	71.047	59.873	158.618	170.065
medium	217.868	223.842	75.005	96.604	292.888	320.462
large	384.381	363.331	163.561	134.228	547.968	497.575
xl	781.901	514.076	323.489	180.433	1105.413	694.529
2.7b	841.362	352.867	456.769	129.098	1298.162	481.981

Table 6: Forward/backward/step means with and without BF16 autocast mixed precision (warmup=5, measured=10, ctx=128).

Mixed precision showed little or negative benefit for the smallest models in this run, but produced clear speedups for larger models. Step-time speedup grew with scale (about $1.10\times$ at large, $1.59\times$ at xl, and $2.69\times$ at 2.7b), consistent with larger workloads benefiting more from Tensor Core throughput and amortizing autocast overhead.

5 Problem: memory_profiling

(a) Memory-profiler instrumentation and timelines

I added memory-profiler support to `cs336.systems/nsys_profile_workload.py` using `torch.cuda.memory._record_dump_snapshot`, and peak-memory JSON export. Snapshots were generated under `results/memory/*.pickle` for successful runs (for example `forward_ctx128_mpooff.pickle` and `forward_ctx128_mpon.pickle`).

From the successful forward profiles, the timeline is consistent with a large rise during the forward region and then a plateau near peak reserved memory after kernel completion. For full train-step runs (forward+backward+optimizer), all 2.7B configurations on this 40GB GPU ran out of memory during optimizer-state allocation, so a complete train-step timeline could not be produced in this environment.

(b) Peak memory by context length

Context length	Forward peak allocated (GiB, FP32)	Full train-step peak (GiB, FP32)
128	17.940	OOM
256	24.176	OOM
512	OOM	OOM

Table 7: Peak memory from profiler JSON outputs for the 2.7B model.

(c) Mixed-precision peak memory

For BF16 autocast mixed precision, forward-pass peak allocated memory was 22.321 GiB (ctx=128), 26.410 GiB (ctx=256), and 36.724 GiB (ctx=512). Full train-step runs with BF16 autocast still OOM at all tested context lengths (128/256/512). In this setup, mixed precision did not significantly reduce memory footprint for the 2.7B model and did not make full train steps fit on a 40GB GPU.

(d) Residual-stream activation size

For one residual-stream activation tensor in FP32, size is

$$\begin{aligned}
 &= \text{batch} \times \text{context} \times d_{\text{model}} \times 4 \text{ bytes} \\
 &= 4 \times L \times 2560 \times 4.
 \end{aligned}$$

So this is $40960L$ bytes, i.e. 5 MB at $L=128$, 10 MB at $L=256$, and 20 MB at $L=512$ (using MB = 1024^2 bytes).

(e) Largest allocations at reduced detail

Inspecting the saved snapshot data for forward runs shows largest active allocations in large chunks (e.g., up to ~ 100 MB blocks in the allocator snapshot, with frequent 40 MB blocks). The available stack traces in this environment resolve mainly to the CUDA caching allocator internals, so the coarse interpretation is that these largest blocks correspond to persistent model/activation storage managed by the allocator during large forward GEMM-heavy regions.

6 Problem: pytorch_attention

TODO: Add benchmark table, memory accounting, and discussion.

7 Problem: torch_compile

TODO: Add compilation benchmarking results and response.

8 Problem: flash_forward

TODO: Add implementation notes and requested deliverables.

9 Problem: flash_backward

TODO: Add backward implementation results and validation.

10 Problem: flash_benchmarking

TODO: Add FlashAttention benchmarking tables and analysis.

11 Problem: distributed_communication_single_node

TODO: Add distributed communication measurements and response.

12 Problem: naive_ddp

TODO: Add naive DDP implementation deliverable details.

13 Problem: naive_ddp_benchmarking

TODO: Add naive DDP benchmarking results and discussion.

14 Problem: minimal_ddp_flat_benchmarking

TODO: Add flat-buffer DDP benchmarking results.

15 Problem: ddp_overlap_individual_parameters

TODO: Add implementation and correctness evidence.

16 Problem: ddp_overlap_individual_parameters_benchmarking

TODO: Add overlap benchmarking results and conclusions.

17 Problem: ddp_overlap_bucketed

TODO: Add bucketed overlap implementation details.

18 Problem: ddp_bucketed_benchmarking

TODO: Add bucket-size benchmarking results and analysis.

19 Problem: communication_accounting

TODO: Add communication-volume accounting calculations.

20 Problem: optimizer_state_sharding

TODO: Add implementation details and validation.

21 Problem: optimizer_state_sharding_accounting

TODO: Add memory/speed accounting and ZeRO comparison.